Thực nghiệm giải N-Queens bằng SAT, CP và MIP đến N=1000

Bùi Đình Cảnh[a]

[a]Sinh viên 24021392, lớp INT3011E_3, Trường Đại học Công nghệ, Đại học Quốc gia Hà Nội; email: buidinhcanh2006@gmail.com

ARTICLE INFO

Keywords:
N-Queens
SAT solving
cardinality encoding
PySAT
constraint programming
mixed-integer programming

Abstract

Bài báo nghiên cứu các mô hình giải chính xác cho bài toán N-Queens, với trọng tâm là ảnh hưởng của kỹ thuật mã hóa ràng buộc at-most-one trong SAT. Năm mã hóa Pairwise, Sequential Counter, Commander, Product và Binary được triển khai trên PySAT/Glucose3 và so sánh với OR-Tools CP-SAT, Gurobi MIP, CPLEX MIP và CPLEX CP Optimizer. Quy trình thực nghiệm có timeout cưỡng bức, checkpoint, kiểm tra nghiệm và phân biệt timeout, giới hạn giấy phép và tiến trình bị hệ điều hành dừng. Bộ dữ liệu gồm 238 lượt chạy trên các kích thước từ $N = 1$ đến $N = 1000$. Tất cả 169 nghiệm SAT thu được đều vượt qua bộ kiểm tra độc lập; 14 lượt UNSAT tại $N = 2, 3$ cũng phù hợp với tính chất bài toán. Trên môi trường Google Colab đã sử dụng, Product là mã hóa duy nhất hoàn thành cả ba lần tại $N = 1000$, với trung vị 28.127 giây, 1,298,508 biến và 11,942,164 mệnh đề. Kết quả cũng cho thấy kích thước CNF không đủ để dự đoán thời gian giải: hiệu năng thay đổi không đơn điệu theo N và phụ thuộc mạnh vào cấu trúc encoding cũng như heuristic của solver.

1. Giới thiệu

Bài toán N-Queens yêu cầu đặt N quân hậu trên bàn cờ có N hàng và N cột sao cho không có hai quân nào cùng hàng, cùng cột hoặc cùng đường chéo. Bài toán có mô tả ngắn nhưng là một ví dụ điển hình để khảo sát ảnh hưởng của mô hình hóa tới hiệu năng solver. Cùng một tập ràng buộc có thể được biểu diễn bằng công thức SAT ở dạng CNF, bằng các ràng buộc toàn cục trong Constraint Programming (CP), hoặc bằng hệ ràng buộc tuyến tính trên biến nhị phân trong MIP.

Mục tiêu của nghiên cứu không phải đếm mọi lời giải, mà tìm một cấu hình hợp lệ hoặc chứng minh không tồn tại cấu hình. Đóng góp chính gồm: (i) triển khai và kiểm thử năm SAT encoding cho AMO; (ii) xây dựng bốn mô hình đối chứng bằng OR-Tools, Gurobi và CPLEX; (iii) xây dựng pipeline thực nghiệm có thể tiếp tục sau khi Colab bị ngắt; và (iv) đánh giá đồng thời thời gian, quy mô encoding và tỷ lệ hoàn thành đến $N = 1000$.

2. Mô hình bài toán

Với mỗi ô (r, c), đặt biến Boolean $x_{r,c}$ bằng 1 khi có hậu tại ô đó. Mô hình SAT và MIP sử dụng các nhóm ràng buộc sau:

$$\sum_{c=0}^{N-1} x_{r,c} = 1 \qquad \text{với mọi hàng } r, \tag{1}$$

$$\sum_{r=0}^{N-1} x_{r,c} = 1 \qquad \text{với mọi cột } c, \tag{2}$$

$$\sum_{(r,c)\ \text{thuộc}\ d} x_{r,c} \le 1 \qquad \text{với mọi đường chéo } d. \tag{3}$$

Trong SAT, ràng buộc bằng 1 được tách thành at-least-one (ALO) và at-most-one (AMO). Với nhóm literal $X = (x_1, \ldots, x_k)$, ALO là clause $(x_1 \text{ OR } \cdots \text{ OR } x_k)$. AMO được thay đổi theo từng encoding. Trong CP, mỗi hàng i dùng một biến nguyên q_i nhận giá trị từ 0 đến $N - 1$ để biểu diễn cột của quân hậu; mô hình chỉ cần ba ràng buộc toàn cục `AllDifferent` trên q_i, $q_i + i$ và $q_i - i$, đúng với mô hình N-Queens được OR-Tools công bố [3].

ORCID(S):

3. Các phương pháp giải

3.1. SAT và các mã hóa AMO

PySAT cung cấp giao diện thống nhất cho solver và các encoding cardinality [7, 6]. Glucose3 được dùng làm SAT backend. Pairwise sinh clause $(\overline{x_i}$ OR $\overline{x_j})$ cho mọi cặp. Sequential Counter dùng biến trạng thái phụ tuyến tính theo kích thước nhóm, dựa trên ý tưởng của Sinz [8]. Binary gán mã bit cho vị trí của literal đúng. Commander chia các literal thành nhóm và tạo biến "chỉ huy" cho mỗi nhóm. Product ánh xạ literal lên một lưới phụ hai chiều và áp dụng AMO cho các chỉ báo hàng/cột.

Pairwise, Sequential và Binary gọi trực tiếp `CardEnc` của PySAT; Commander và Product được cài đặt trong project. ALO được áp dụng cho mọi hàng và cột; AMO được áp dụng cho hàng, cột và đường chéo. Bộ kiểm thử truth-table kiểm tra trực tiếp rằng cả năm cài đặt AMO chấp nhận đúng các phép gán có không quá một literal đúng. Ngoài ra, Glucose3 là solver CDCL có heuristic học clause; những đặc trưng này giải thích vì sao số clause không ánh xạ đơn giản thành thời gian chạy [1].

3.2. CP-SAT và CP Optimizer

Mô hình OR-Tools CP-SAT và CPLEX CP Optimizer đều dùng N biến nguyên cùng ba ràng buộc `AllDifferent`. CP-SAT là solver CP trên nền SAT cho biến nguyên [2]; CP Optimizer cung cấp các ràng buộc toàn cục cho bài toán tổ hợp [5]. Mỗi solver được giới hạn một worker để giảm sai lệch do mức độ song song khác nhau.

3.3. Gurobi MIP và CPLEX MIP

Hai mô hình MIP dùng N^2 biến nhị phân và các phương trình/bất phương trình ở phần trước. Đây là bài toán khả thi, vì vậy không cần hàm mục tiêu. Gurobi và CPLEX được cấu hình một thread, seed theo lần lặp và time limit nội bộ. Gurobi tuân theo mô hình formulate–solve–query của API tối ưu toán học [4]; CPLEX MIP dùng cùng mô hình để tạo phép so sánh giữa hai solver thương mại.

4. Thiết kế thực nghiệm

Thực nghiệm chạy trên Google Colab với 2 CPU logic, một worker cho mỗi solver và bốn phase:

- smoke: $N = 1, 2, 3, 4, 8$, một lần, timeout 15 giây;
- standard: $N = 16, 32, 64, 128$, năm lần, timeout 60 giây;
- CPLEX: $N = 8, 16, 32$, ba lần, timeout 60 giây;
- large: $N = 256, 512, 1000$, ba lần, timeout 120 giây.

Môi trường ghi nhận Linux, Python 3.13.16, PySAT 0.1.8.dev9, OR-Tools 9.15.6755, Gurobi 13.0.3 và CPLEX 22.1.2.1. Pairwise không được chạy ở phase large vì số clause tăng mạnh; Gurobi cũng không chạy ở phase này vì license giới hạn đã xuất hiện từ $N = 64$.

`Total_Time` gồm thời gian xây dựng mô hình/encoding và thời gian giải, nhưng không gồm chi phí khởi tạo process cha. `Wall_Time` bao gồm cả khởi tạo process và IPC. Với Glucose3, seed trong script không được truyền vào solver; vì vậy các lần lặp SAT đo độ dao động thời gian trên cùng cấu hình tìm kiếm mặc định, không phải các lần chạy ngẫu nhiên độc lập.

5. Kết quả và thảo luận

5.1. Tính đúng đắn và mức độ hoàn thành

Bộ dữ liệu có 238 run, không có khóa $(phase, N, method, run)$ bị trùng. Tổng hợp trạng thái được trình bày trong Bảng 1. Tất cả 169 nghiệm SAT đều có `Solution_Valid=True`; 14 kết quả UNSAT đều thuộc $N = 2$ và $N = 3$. Không có `INVALID_SOLUTION`.

Hình 1 thể hiện số lần thành công trên tổng số lần chạy. Cần đọc hình này cùng đồ thị thời gian: trung vị chỉ được tính trên các run thành công, nên một điểm thời gian không biểu diễn xác suất hoàn thành.

Table 1
Phân bố trạng thái của toàn bộ lượt chạy.

Trạng thái	Số lượt
SAT	169
UNSAT	14
TIMEOUT	39
LICENSE_LIMIT	13
KILLED	3
Tổng	238

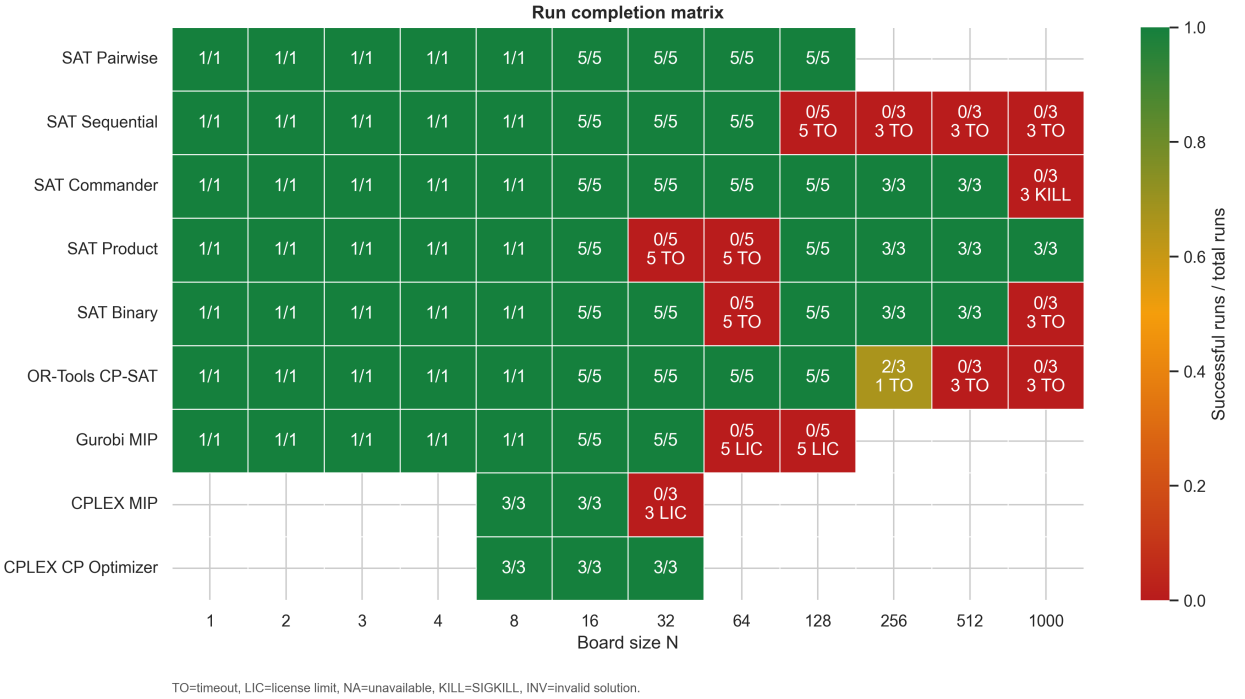


Figure 1: Ma trận hoàn thành. TO: timeout; LIC: giới hạn license; KILL: tiến trình nhận SIGKILL. Ô trống là cấu hình không được đưa vào phase đó.

5.2. Thời gian giải

Hình 2 và Bảng 2 cho thấy không có một phương pháp thắng ở mọi kích thước. Sequential nhanh nhất tại $N = 16$, Gurobi nhanh nhất tại $N = 32$, OR-Tools dẫn đầu tại $N = 64$, còn Product tốt nhất ở $N = 128$, 256 và 1000. Tại $N = 512$, Binary có trung vị thấp nhất. Các kết luận này chỉ áp dụng cho môi trường và tham số đã ghi nhận.

Bảng 3 tập trung vào SAT. Product timeout ở $N = 32, 64$ nhưng hoàn thành cả ba lần ở $N = 1000$; Binary timeout tại $N = 64$ nhưng hoàn thành tại $N = 128, 256, 512$. Đây là bằng chứng thực nghiệm rằng độ khó tìm kiếm không tăng đơn điệu theo N đối với một encoding/heuristic cố định. Không nên diễn giải hiện tượng này thành bảo đảm rằng Product luôn tốt ở kích thước lớn.

OR-Tools hoàn thành 2/3 lần ở $N = 256$ với thời gian thành công khoảng 40–43 giây, sau đó timeout hoàn toàn tại $N = 512, 1000$. CPLEX CP hoàn thành 3/3 lần tại $N = 8, 16, 32$; CPLEX MIP gặp giới hạn license ở $N = 32$. Gurobi gặp giới hạn license từ $N = 64$. Các giá trị `LICENSE_LIMIT` phản ánh môi trường cấp phép, không phải thất bại thuật toán.

Ba run Commander tại $N = 1000$ kết thúc với exit code -9, tức SIGKILL. Điều này phù hợp với tình huống process bị hệ thống Colab dừng do áp lực tài nguyên, nhưng không có log kernel để khẳng định chắc chắn nguyên nhân là OOM; do đó dữ liệu được gắn nhãn thận trọng là `KILLED`.

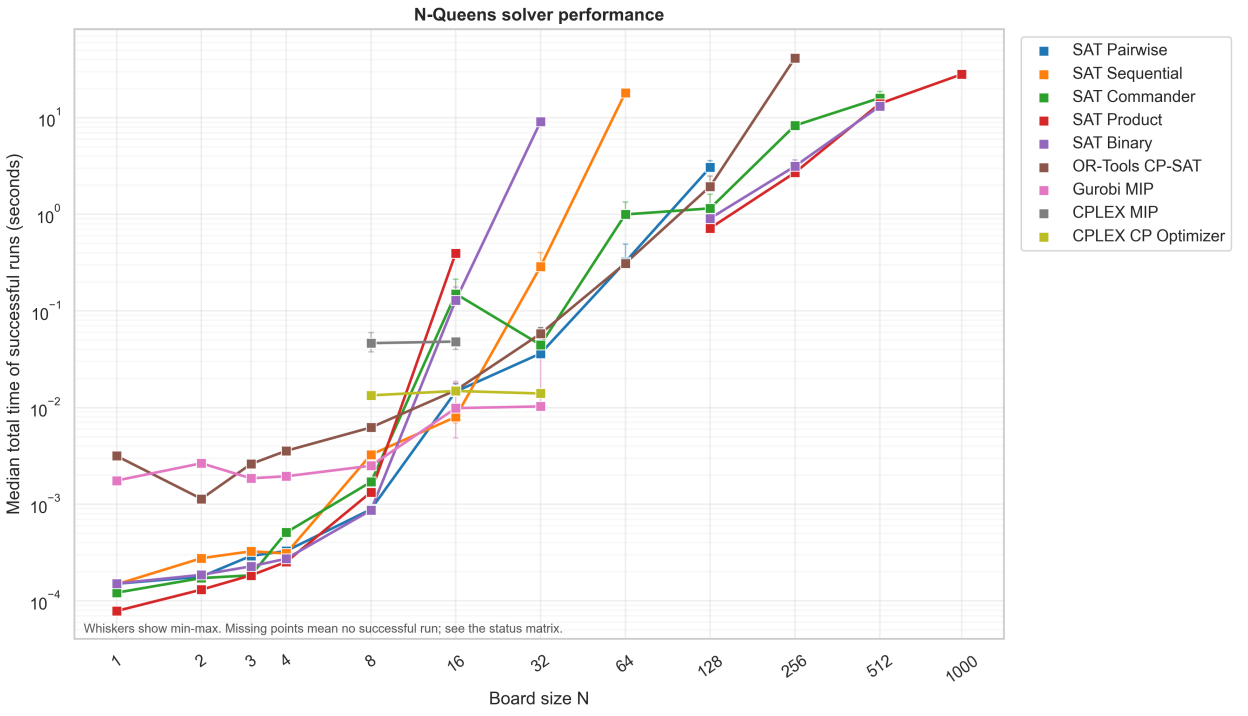


Figure 2: Trung vị thời gian của các run thành công; whisker là min–max. Đường không được nối qua kích thước không có run thành công.

Table 2
Phương pháp có trung vị thấp nhất trong các cấu hình đã chạy.

N	Phương pháp nhanh nhất	Trung vị (s)	Thành công
16	SAT Sequential	0.0080	5/5
32	Gurobi MIP	0.0103	5/5
64	OR-Tools CP-SAT	0.3113	5/5
128	SAT Product	0.7192	5/5
256	SAT Product	2.7036	3/3
512	SAT Binary	13.1581	3/3
1000	SAT Product	28.1271	3/3

5.3. Quy mô SAT encoding

Hình 3 và Hình 4 cho thấy Pairwise không dùng biến phụ nhưng số clause tăng nhanh. Product tạo nhiều biến hơn nhưng có số clause thấp hơn rõ rệt ở các kích thước lớn quan sát được. Tại $N = 1000$, Product có 1,298,508 biến và 11,942,164 clause, vẫn hoàn thành với trung vị 28.127 giây. Các run timeout/killed không trả lại thống kê encoding, vì vậy đồ thị dừng đường tại các điểm thiếu thay vì nội suy qua chúng.

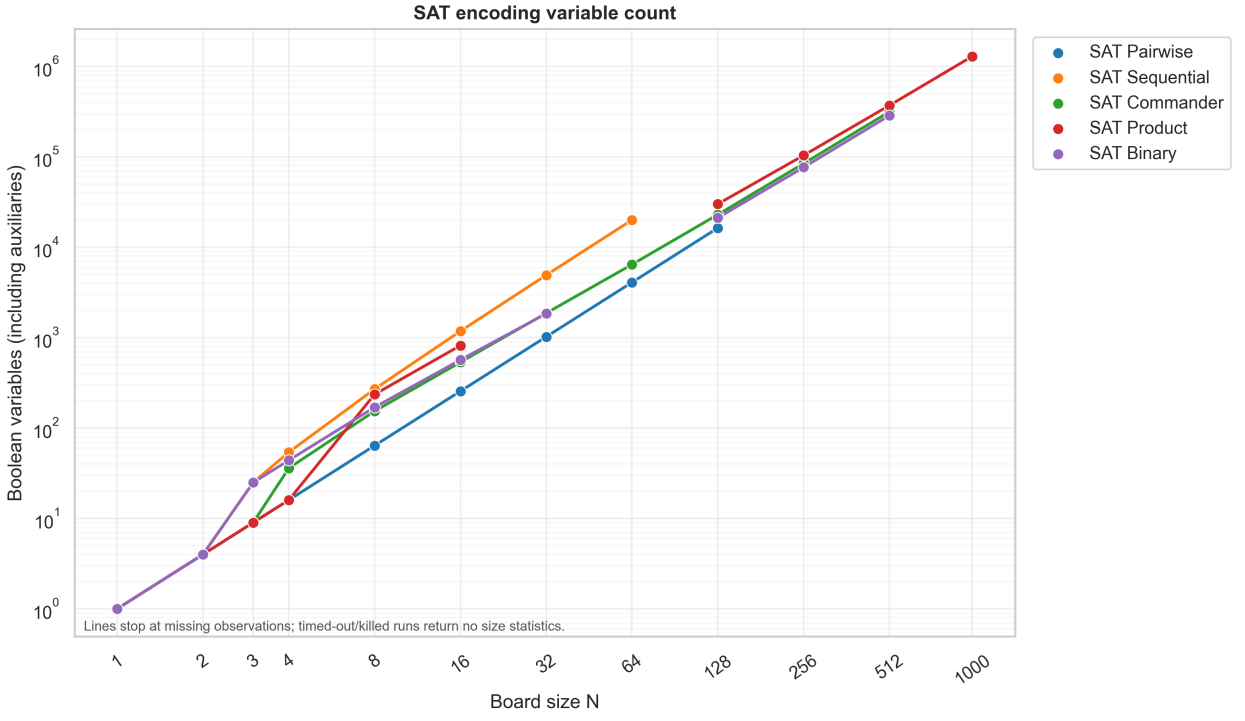


Figure 3: Số biến Boolean, bao gồm biến phụ, của các SAT encoding.

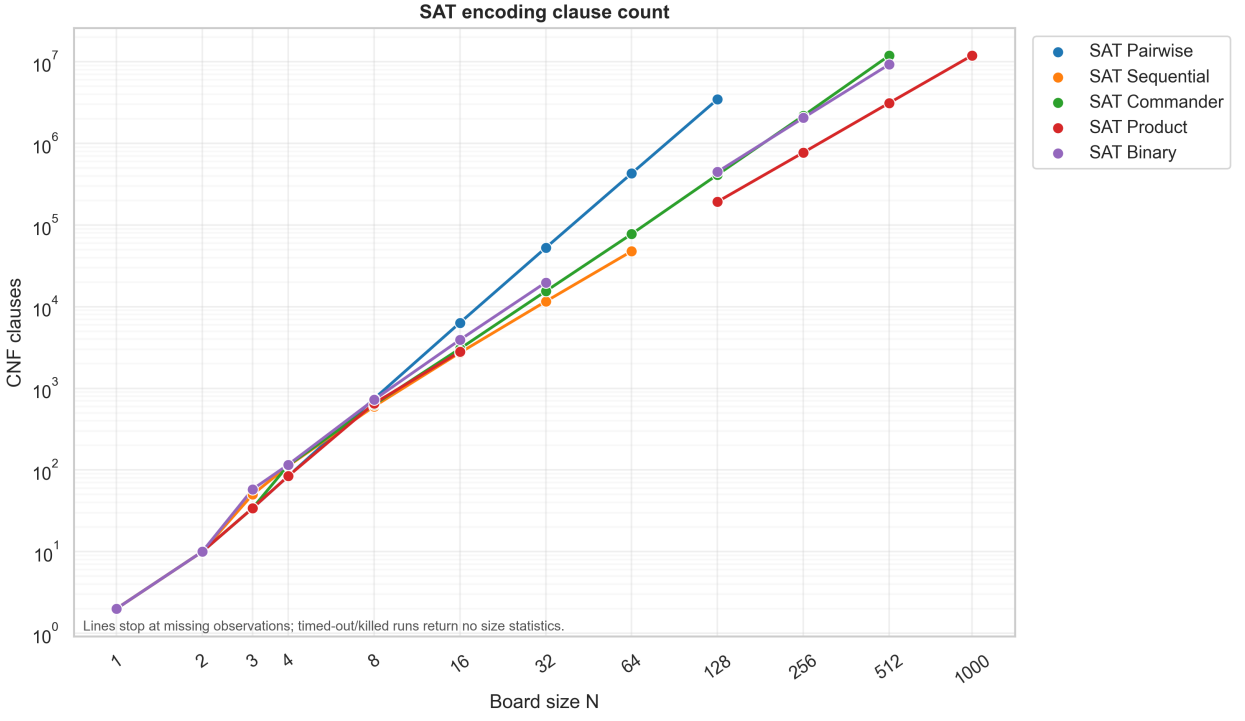


Figure 4: Số clause CNF của các SAT encoding.

Table 3
Thời gian trung vị SAT theo giây; trong ngoặc là số run thành công/tổng số run. TO: timeout; KILL: SIGKILL; "–": không chạy.

N	Pairwise	Sequential	Commander	Product	Binary
16	0.015 (5/5)	0.008 (5/5)	0.151 (5/5)	0.397 (5/5)	0.129 (5/5)
32	0.036 (5/5)	0.288 (5/5)	0.045 (5/5)	TO	9.090 (5/5)
64	0.322 (5/5)	18.143 (5/5)	0.998 (5/5)	TO	TO
128	3.063 (5/5)	TO	1.153 (5/5)	0.719 (5/5)	0.908 (5/5)
256	–	TO	8.309 (3/3)	2.704 (3/3)	3.147 (3/3)
512	–	TO	16.003 (3/3)	14.004 (3/3)	13.158 (3/3)
1000	–	TO	KILL	28.127 (3/3)	TO

6. Hạn chế và nguy cơ ảnh hưởng tính hợp lệ

Thứ nhất, Colab là môi trường chia sẻ; mô hình CPU và tổng RAM chưa được lưu trong bộ metadata cũ, nên không nên so sánh trị tuyệt đối với máy khác. Thứ hai, số lần lặp khác nhau giữa các phase (1, 3 hoặc 5), và SAT không dùng seed ngẫu nhiên. Thứ ba, timeout ở process cha làm mất số biến/clause và không cho biết run dừng trong encoding hay solving. Thứ tư, giới hạn license làm thu hẹp miền kích thước của Gurobi và CPLEX MIP. Cuối cùng, nghiên cứu chỉ tìm một lời giải khả thi; kết quả không đại diện cho bài toán đếm toàn bộ nghiệm N-Queens.

7. Kết luận

Project đã triển khai đầy đủ các nhóm phương pháp được yêu cầu và xác minh tính đúng đắn trên các trường hợp SAT/UNSAT nhỏ. Thực nghiệm đến $N = 1000$ cho thấy việc chọn encoding có ảnh hưởng quyết định: Pairwise có ít biến nhưng bùng nổ clause, Sequential suy giảm mạnh từ $N = 64$, trong khi Product và Binary mở rộng tốt hơn ở một số kích thước lớn. Tuy nhiên, các bất thường không đơn điệu nhấn mạnh rằng không thể đánh giá encoding chỉ bằng số biến hoặc số clause; cần báo cáo đồng thời thời gian, tỷ lệ hoàn thành và giới hạn môi trường.

Data availability

Mã nguồn, unit test, hướng dẫn Colab, dữ liệu thô, bảng tổng hợp và biểu đồ được lưu tại `https://github.com/WilliamsBui-fnd/INT3011E_3`. File `benchmark_raw.csv` được xem là dữ liệu gốc; mọi bảng và hình trong báo cáo có thể tái tạo bằng `combine_results.py`, `plot_results.py` và `prepare_report.py`.

Tuyên bố về công cụ AI hỗ trợ

Trong quá trình chuẩn bị project, tác giả sử dụng ChatGPT/Codex để hỗ trợ rà soát mã nguồn, tổ chức thí nghiệm, trực quan hóa và hiệu đính ngôn ngữ. Tác giả đã kiểm tra mã, nghiệm, số liệu và chịu trách nhiệm cuối cùng về nội dung báo cáo.

References

[1] Audemard, G., Simon, L., 2009. Predicting learnt clauses quality in modern sat solvers, in: Proceedings of the 21st International Joint Conference on Artificial Intelligence, pp. 399–404.

[2] Google, a. Cp-sat solver. `https://developers.google.com/optimization/cp/cp_solver`. OR-Tools documentation, truy cập ngày 11-10-2026.

[3] Google, b. The n-queens problem. `https://developers.google.com/optimization/cp/queens`. OR-Tools documentation, truy cập ngày 11-10-2026.

[4] Gurobi Optimization, LLC, 2026. Gurobi optimizer reference manual, version 13.0. `https://docs.gurobi.com/projects/optimizer/en/current/`.

[5] IBM, . Ibm ilog cp optimizer overview. https://www.ibm.com/docs/en/icos/22.1.0?topic=cpco-overview. Truy cập ngày 11-10-2026.
[6] Ignatiev, A., Morgado, A., Marques-Silva, J., 2018. Pysat: A python toolkit for prototyping with sat oracles, in: Theory and Applications of Satisfiability Testing – SAT 2018, Springer. pp. 428–437.
[7] PySAT developers, . Pysat documentation. https://pysathq.github.io/docs/html/. Truy cập ngày 11-10-2026.
[8] Sinz, C., 2005. Towards an optimal cnf encoding of boolean cardinality constraints, in: Principles and Practice of Constraint Programming – CP 2005, Springer. pp. 827–831. doi:10.1007/11564751_73.